SDC HACKATHON 2026 · TECHNICAL TOOL USAGE EVIDENCE

RunPod Usage Proof

Serverless GPU endpoint for model inference, and the recorded shutdown of that endpoint

Team	Falcons	**Institution**	Xiamen University Malaysia
Submission	KiraOrder — multilingual order intake for wholesale clerks	**Team lead**	ZHOU JINGXI
Tool declared	RunPod Serverless (GPU inference)	**Account holder**	Xiong (team member)
Evidence captured	24 October 2026, 20:25 (UTC+08:00)	**Document issued**	25 October 2026, Rev. 2
Evidence type	RunPod console screenshots of one serverless endpoint — Overview, Workers and Logs tabs — reproduced without alteration		

ENDPOINT	MODEL	GPU	RUNNING WORKERS
Ready	**Qwen2.5-7B**	**1 × 24 GB**	**0**
lx22t9308hc10c	AWQ, OpenAI-compatible	PRO 6000 MIG	$0.00000 / s

0 Purpose and coverage

This document records that team Falcons deployed and operated a language-model inference endpoint on RunPod, and that the endpoint was in a zero-worker state, with no per-second charge accruing, when the console was captured. Three screenshots of the same endpoint are reproduced: its configuration, its worker and spend state, and its shutdown log.

Two boundaries are set early. The screenshots evidence the **platform**: that an endpoint of this shape existed and that it was not consuming GPU time at capture. They do not evidence model quality, and they do not evidence end-to-end behaviour of the public application. The shutdown log is likewise read narrowly: it shows that the application reported shutdown completion, and it is deliberately **not** used to claim that in-flight requests were allowed to finish (Section 3).

> **Summary of what is proven**
>
> A RunPod serverless endpoint named **kiraorder-qwen** served **Qwen/Qwen2.5-7B-Instruct-AWQ** through the **vllm/vllm-openai:latest** container on a single **24 GB** GPU, was reachable at **https://lx22t9308hc10c.api.runpod.ai**, and logged **shutdown triggered at 20:25:09.711** and **application shutdown complete at 20:25:10.339** (UTC+08:00) on 24 October 2026. At capture the endpoint reported **0 running workers** and **$0.00000/s**.

CONTENTS

0	Purpose and coverage	Scope statement	1	Endpoint configuration	Figure 1 · overview
2	Workers and spend rate	Figure 2 · workers	3	Shutdown log	Figure 3 · logs
4	Evaluation evidence and latency	Datasets	5	Role in the architecture	Layer table
6	Account, credits and cost	Billing summary	7	Scope and limitations	Boundaries
8	Evidence inventory	File list	9	Cross-references	Where else declared

1 Endpoint configuration

Figure 1 is the Overview tab of the serverless endpoint in the RunPod console. The endpoint is a load-balancing endpoint whose requests are distributed across available workers; the request URL, endpoint identifier, container image, model and GPU allocation are all visible on the same screen.

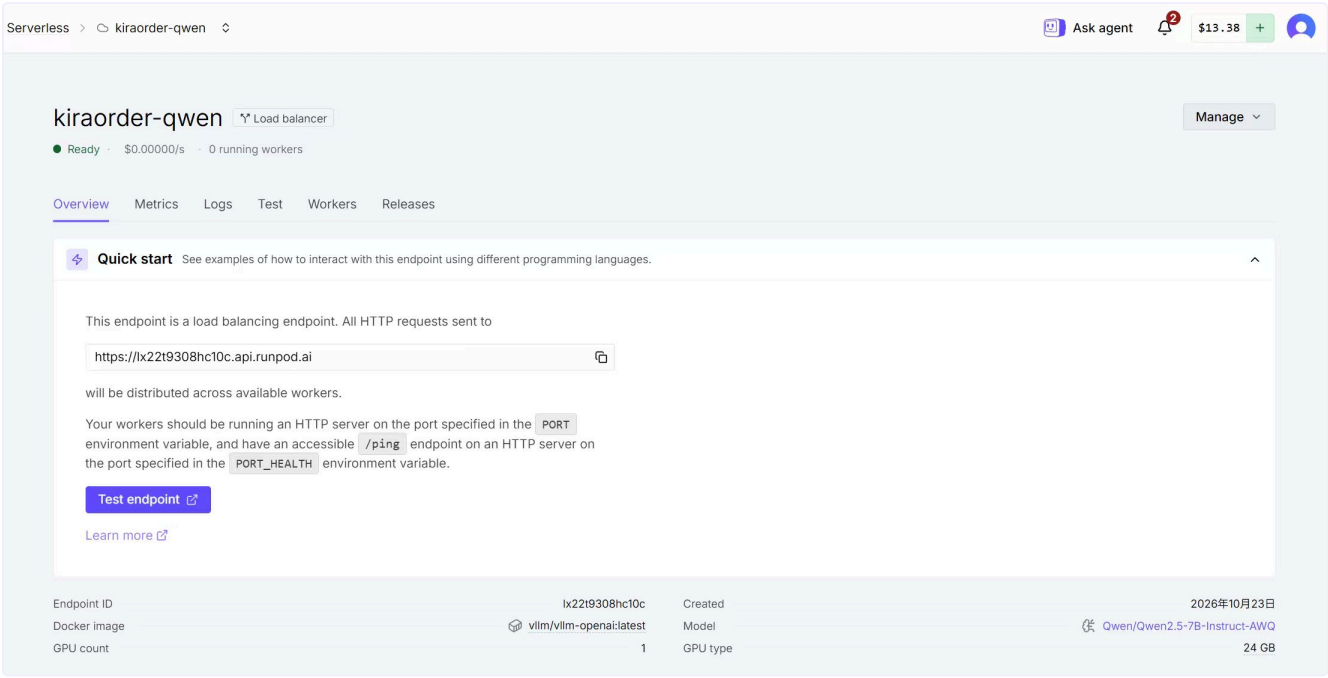


FIGURE 1 **RunPod — kiraorder-qwen, Overview tab.** Header shows status *Ready*, spend rate *$0.00000/s* and *0 running workers*. The Quick start panel exposes the request URL *https://lx22t9308hc10c.api.runpod.ai*. The metadata strip at the bottom reads Endpoint ID *lx22t9308hc10c*, Docker image *vllm/vllm-openai:latest*, model *Qwen/Qwen2.5-7B-Instruct-AWQ*, created *2026-10-23*, GPU count *1*, GPU type *24 GB*.

Field on screen	Value as displayed	Reading
Endpoint name	kiraorder-qwen	Project-specific serverless endpoint (load balancer)
Endpoint ID	lx22t9308hc10c	Unique identifier of the deployed endpoint
Endpoint URL	https://lx22t9308hc10c.api.runpod.ai	Public request URL referenced by the backend's model branch
Created	2026-10-23	Endpoint created the day before the evaluation session
Docker image	vllm/vllm-openai:latest	vLLM server exposing an OpenAI-compatible HTTP API
Model	Qwen/Qwen2.5-7B-Instruct-AWQ	7B instruction-tuned model, AWQ 4-bit quantised
GPU count	1	Single-GPU endpoint
GPU type	24 GB	24 GB-class GPU partition

The console labels the GPU by memory size only. The specific partition type — PRO 6000 MIG, 24 GB, one GPU, region US-NE-1 — is the value declared in the Technical Tools Declaration and reported by the operator; it is not printed on this screen. No API key or Bearer token appears in any screenshot.

2 Running workers and spend rate

Figure 2 is the Workers tab of the same endpoint, captured in the same session. The endpoint header is the reliable place to read the live worker count and the current spend rate, and both read zero at capture time.

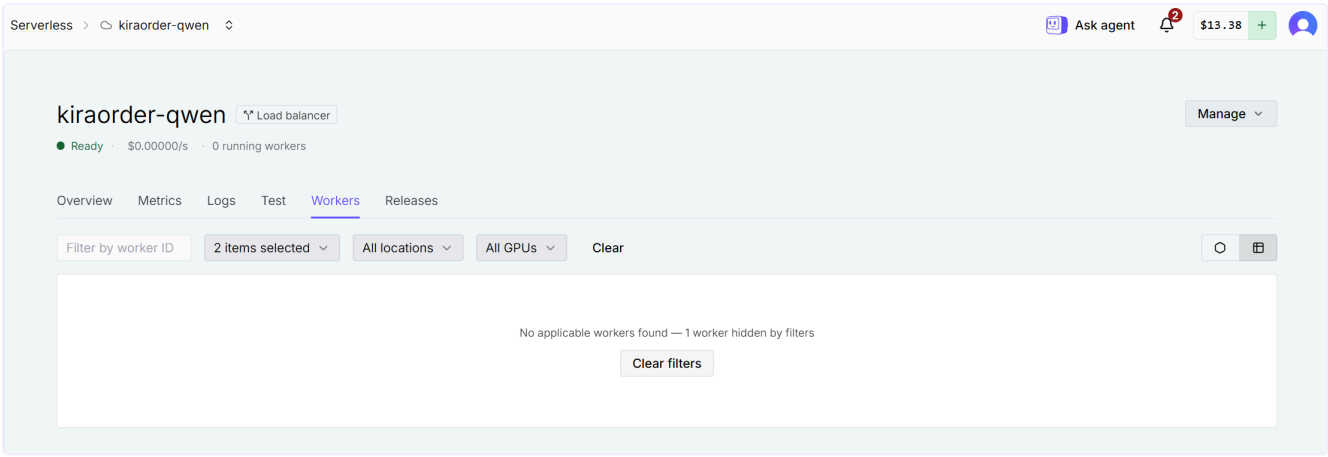


FIGURE 2 **RunPod — Workers tab, zero-worker state.** Header reads status *Ready*, *$0.00000/s* and *0 running workers*; account balance shows *$13.38*. The table area reports no applicable workers for the current filter selection. The filter row indicates filters were applied (two filter chips plus an "All GPUs" selection), so the header metric — not the empty table — is the value relied on here.

Field on screen	Value as displayed	Reading
Endpoint status	Ready	Endpoint configuration healthy and able to accept requests
Running workers	0 running workers	No GPU worker allocated at capture time
Spend rate	$0.00000/s	No per-second GPU charge accruing at capture time
Account balance	$13.38	Remaining RunPod credit on the account
Workers table	"No applicable workers found — 1 worker hidden by filters"	Filtered view; see the note below

A note on the filtered Workers table — stated rather than smoothed over

The Workers table in Figure 2 does not show an empty worker list in its default state; it shows a filtered view that hides one worker record. We therefore do not cite the table as proof of zero workers. The claim rests on the endpoint header metric, which is computed by RunPod independently of any client-side filter and reads *0 running workers* and *$0.00000/s* in both Figure 1 and Figure 2, in two separate tabs. What this establishes is the state **at capture**; it is not a statement about how the endpoint behaved between requests during the evaluation session (Section 4).

3 Shutdown log

Figure 3 is the Logs tab at the end of the session. It records a shutdown request, the teardown of the engine and the API server, and a final completion line.

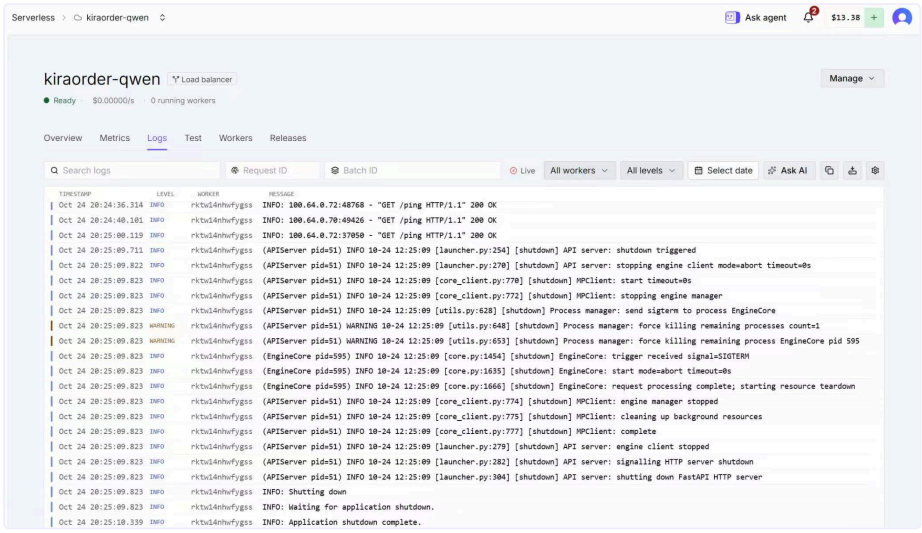


FIGURE 3 **RunPod — Logs tab, shutdown sequence.** At *20:25:09.711* the API server logs *"[shutdown] API server: shutdown triggered"*. The engine client and engine manager are stopped, a SIGTERM is sent to EngineCore, and at *20:25:10.339* the final line *"INFO: Application shutdown complete."* is written. The three `/ping` requests at the top of the panel are load-balancer health checks.

Console timestamp (UTC+08:00)	Log line	What it does and does not show
20:25:09.711	[shutdown] API server: shutdown triggered	A shutdown was requested. Says nothing about requests in flight
20:25:09.822–823	stopping engine client … stopping engine manager … send sigterm to EngineCore … force killing remaining process EngineCore pid 595	Engine teardown was initiated and the engine process terminated by force
20:25:09.823	start mode=abort timeout=0s / request processing complete	Abort-mode teardown with a **zero-second** grace period — not a drain-and-wait
20:25:09.823	MPClient: engine manager stopped — MPClient: complete	`MPClient` (vLLM's multiprocessing client, not MPI) reports the engine manager stopped
20:25:09.823	API server: shutting down FastAPI HTTP server	The public HTTP surface was closed
20:25:10.339	**INFO: Application shutdown complete.**	Shutdown completion reported — the only conclusion this page draws

> **How far this log can be read — a correction**
>
> An earlier revision called this a graceful shutdown with requests drained and workers released cleanly. The log does not support that: `mode=abort` with `timeout=0s`, then `force killing remaining process EngineCore pid 595`. The defensible claim is narrower — **a shutdown was requested and completion reported 0.63 s later**. No claim is made about request completion, drainage or worker cleanliness.

Row timestamps are local (`10-24 20:25:09`); the embedded messages are UTC (`INFO 10-24 12:25:09`) — the same instant.

4 Evaluation evidence and latency

The endpoint was used for the team's AI-only evaluation runs. The datasets and evidence files are named explicitly below, because "tested on a frozen evaluation set" is too loose a description to check.

Dataset / run	Evidence file	What it covers
dev_41 — AI-only production run	`dev41_ai_only_v1_prod_2026-10-24.annotated.json`	23 cases executed once each in live LLM mode against `app/ai@db6f0ce`. 23/23 model calls, **fallback 0**. Accuracy is **NOT_COMPUTED** because the inputs have no independent answer key
He's 6-case acceptance	`he_6cases_final_2026-10-24.txt`	Six contract cases on the same code base: **5 PASS / 1 KNOWN_LIMITATION** (normal.json = MODEL_SPAN_TRUNCATED)
UOM check within dev_41	same annotated file	**24 of 31** unit-of-measure cases correct, with 7 known gaps: 3 alias-injection gaps (*botol*, *pail*) and 4 bulk-*kg* regressions
TEST32 — separate frozen set	`TEST32_SCOPE-2026-10-24-v4`, freeze manifest	**Not offered as RunPod evidence.** The set holds 32 items / 34 executions, 12 of which are E2E/workflow items excluded from the AI-only scope. It was **not fully executed** against this endpoint, and this document does not imply that it was

4.1 Cold start and warm latency are separate measurements

Two latency figures exist in the project records and they measure different things. They are listed separately here so that neither can be read as a general per-request timing.

Measurement	Value	Source and status
Cold start — scale-from-zero to first `/health`	**≈198.5 s**	`docs/handoff_2026-10-23.md`, Section 1.2 (23 October 2026). A **historical** measurement of one scale-from-zero event, not a current availability guarantee
Hot-state first call	1.87 s	`runpod_benchmark_2026-10-23.txt`. The handoff explicitly identifies this as a **hot-worker first call**, not a cold start

> **What is not claimed about scaling**
>
> An earlier revision of this page stated that the endpoint was scaled to zero between runs, which would imply a cold start before every request. The evidence does not support that. What is documented is (i) a zero-worker state **at the moment of capture** (Section 2) and (ii) **one** recorded scale-from-zero event lasting about 198.5 s (the 23 October handoff). The endpoint is configured to scale to zero when idle; how often that happened during the evaluation is not recorded, and no per-request cold-start claim is made.

5 Role of RunPod in the KiraOrder architecture

RunPod supplies the GPU inference layer for the configured model branch. It is one path among several: exact catalogue aliases resolve without a model call, and the public deployment falls back while no worker is active.

Layer	Hosted on	What it does
Frontend — primary	GitHub Pages hozier123.github.io/sdc-2026-team-project/	React / TypeScript clerk interface. The primary entry point a reviewer should open
Frontend — legacy	Replit kiraorder-web.replit.app	Earlier build of the same interface, kept for reference; no longer the primary frontend
Backend API	Replit kiraorder-api.replit.app	FastAPI service, versioned under `/api/v1`, protected by a Bearer token
Database	Neon PostgreSQL	Workflow records, approvals and audit trail
Model serving	RunPod Serverless lx22t9308hc10c	vLLM serving Qwen2.5-7B-Instruct-AWQ behind an OpenAI-compatible API, used for the evaluation runs in Section 4

How the endpoint was used

- **Extraction evaluation.** Masked order text from the *dev_41* set was sent to the endpoint to test structured field extraction; results are the evidence files in Section 4.
- **Acceptance sampling.** The six contract cases were run as an acceptance check, reported as 5 PASS / 1 KNOWN_LIMITATION.
- **Latency characterisation.** One scale-from-zero event was timed end to end; the benchmark's first call was recorded in a warm state. The two are reported separately, not averaged.
- **Cost control.** The endpoint was returned to a zero-worker state after the session; the capture in Section 2 shows the resulting spend rate.

6 Account, credits and cost

Item	Value	Note
Credit source	Sponsor-provided	RunPod credits supplied through the hackathon sponsor programme, as declared in the tool inventory
Balance at capture	$13.38	Visible in the console header of Figures 1–3
Rate at capture	$0.00000 / s	Zero because no worker was allocated
GPU allocated at capture	None	Zero-worker state; see Section 2
Shutdown logged	20:25:09 → 20:25:10, 24 Oct 2026	Completion line at 20:25:10.339; see Section 3

7 Scope and limitations

These screenshots are platform-usage evidence. They are deliberately not presented as evidence of model quality or of a finished end-to-end pipeline, and the following statements bound what they can be used to support.

Statement	Status	Detail
RunPod endpoint deployed and configured	EVIDENCED	Figure 1 — endpoint ID, URL, image, model, GPU size
Zero workers, no GPU billed at capture	EVIDENCED	Figures 1 and 2 — header metrics
Application shutdown reported complete	EVIDENCED	Figure 3 — trigger line and completion line
Requests were drained before shutdown	NOT CLAIMED	The log shows `mode=abort timeout=0s` and a force-kill of EngineCore pid 595. Read narrowly, see Section 3
Full AI acceptance passed	NOT CLAIMED	No accuracy claim: 23 live cases with zero fallback, accuracy NOT_COMPUTED, UOM 24/31
Public app performs live model inference	NOT CLAIMED	`AI_PROVIDER=runpod_adapter` is configured, but no worker was active at capture, so public parse falls back. The earlier stub configuration is historical
Per-request cold start	NOT CLAIMED	One scale-from-zero event is recorded (≈198.5 s); no claim is made that every request followed a scale-to-zero
Worker count from the Workers table	QUALIFIED	The table view was filtered in Figure 2; the zero-worker claim relies on the endpoint header metrics in two tabs
TEST32 executed against this endpoint	NOT CLAIMED	32 items / 34 executions, 12 E2E items outside AI-only scope; not fully executed and not offered as RunPod evidence
Sustained throughput or cost per order	NOT MEASURED	No aggregate throughput or per-order cost figure is claimed from these screens

Notes on reproduction

The screenshots are reproduced without retouching. The only processing applied was cropping at the image boundary; no value, label or log line was altered, added or removed. The shutdown log is reproduced down to its completion line.

8 Evidence inventory

File	Console tab	Content
runpod-overview-2026-10-24	Overview	Endpoint ID, URL, image, model, GPU count and size, created date
runpod-workers-zero-2026-10-24	Workers	0 running workers, $0.00000/s, balance $13.38
runpod-logs-shutdown-2026-10-24	Logs	Shutdown trigger 20:25:09 → application shutdown complete 20:25:10
This document	—	Consolidated proof, field-by-field reading, evaluation file references and limitation statement

9 Cross-references in the submission

The same facts appear in the team's other submission documents. These pointers let a reviewer check one against the other rather than relying on this document alone.

Fact in this document	Also recorded in
Endpoint ID, request URL and load-balancer role	Technical Tools Declaration — RunPod Serverless Load Balancer entry in the tool inventory
Container image, model, GPU count and size	Technical Tools Declaration — Appendix C, RunPod deployment evidence
Shutdown sequence of 24 October	Technical Tools Declaration — Appendix D, RunPod shutdown evidence
23-case evaluation, zero fallback, UOM 24/31	Technical Tools Declaration — evaluation summary; evidence file named in Section 4
Cold start of about 198.5 s	`docs/handoff_2026-10-23.md` Section 1.2, and the cold-start source note in the Technical Tools Declaration
Model branch and the Replit backend it sits behind	Replit Usage Proof (companion document), Sections 2–3

Attestation

The undersigned confirms that the screenshots reproduced in this document were captured from the vendor console on the date stated, that the readings in the tables correspond to those screens, and that the limitations and gaps listed above are complete to the best of the team's knowledge. Where a value is declared rather than evidenced, it is marked as such.

Issued by team **Falcons**, Xiamen University Malaysia, for SDC Hackathon 2026. Revision 2, 25 October 2026.

ZHOU JINGXI — Team lead | DSC2509065@xmu.edu.my | 25 October 2026